

LINGUAGEM DE PROGRAMAÇÃO (PYTHON)

Aulas 01 e 02 • Fundamentos, sintaxe, variáveis e entrada de dados

Curso Técnico de Nível Médio em Informática • CETAM

Onde estamos no curso

Componente: Linguagem de Programação (Python)

Carga horária: 60 horas — 48h presenciais + 12h de atividades não presenciais.

Objetivo do componente: desenvolver aplicações computacionais de pequeno porte.

Nesta primeira etapa: construir uma base sólida antes de avançar para decisões, repetições, funções, estruturas de dados e POO.



A proposta segue a matriz e os objetos de conhecimento do PPC.

Como executar um programa Python

Editor/IDE → escrever o código.

Interpretador Python → executa o código.

Terminal → permite executar arquivos .py.

Exemplo: `python programa.py`

```
arquivo: aula01.py
```

```
print("Olá, turma!")
```

```
$ python aula01.py
```

Primeiro programa

A função `print()` exibe informações na saída.

Textos normalmente são escritos entre aspas.

Podemos exibir texto, números e resultados de expressões.

```
print("Olá, mundo!")  
print("CETAM")  
print(10 + 5)
```

Comentários

Comentários ajudam a explicar o código para humanos.
Em Python, um comentário de uma linha começa com #.
Comentários não são executados como instruções.

```
# Meu primeiro programa  
print("Python")  
  
# Calculando uma soma  
print(10 + 20)
```

Indentação: atenção!

Python utiliza indentação para organizar blocos de código.

No início, vamos manter 4 espaços como padrão.

Mesmo antes de estudar if/for/while, é importante criar o hábito de organizar o código.

```
if True:  
    print("Bloco indentado")
```

Variáveis

Uma variável é um nome associado a um valor.

A atribuição usa o operador =.

O valor pode mudar durante a execução.

Escolha nomes claros e significativos.

```
nome = "Ana"  
idade = 18  
altura = 1.65
```

```
print(nome)  
print(idade)
```

Tipos de dados básicos

str → texto

int → números inteiros

float → números com casas decimais

bool → True ou False

A função `type()` permite verificar o tipo de um valor.

```
nome = "Ana"      # str
idade = 18         # int
altura = 1.65      # float
aprovado = True    # bool

print(type(idade))
```

Boas práticas para nomes

Prefira nomes que expressem o significado do dado.

Use snake_case para variáveis: nome_completo, salario_mensal.

Evite nomes genéricos como x, y, a, b quando não houver motivo.

Não use palavras reservadas da linguagem.

```
nome_aluno = "João"  
salario_mensal = 2500.00  
quantidade_itens = 10
```

Prática guiada

Crie um programa que apresente seu nome, curso, cidade e uma frase sobre o que espera aprender em Python.

Depois, crie três variáveis: idade, altura e nome.

Exiba os valores e seus respectivos tipos usando `type()`.

Altere um dos valores e observe o resultado.

Exercícios

1. Programa de apresentação.
2. Dados de um aluno: nome, idade, curso e nota.
3. Cadastro de computador: marca, modelo, memória e armazenamento.
4. Identifique o tipo de cada valor apresentado pelo professor.
5. Corrija um programa com erros de sintaxe, nomes e organização.

Entrada de dados

Até agora, os valores foram definidos diretamente no código.

Agora o programa poderá receber informações do usuário.

A função `input()` sempre retorna texto (`str`).

Para cálculos, normalmente precisamos converter o valor.

```
nome = input("Digite seu nome: ")
print("Olá,", nome)
```

Conversão de tipos

`int()` → converte para inteiro quando possível.

`float()` → converte para número decimal quando possível.

`str()` → converte para texto.

A conversão deve ser feita de acordo com a operação que queremos realizar.

```
idade = int(input("Idade: "))
salario = float(input("Salário:
                    "))
print(idade)
print(salario)
```

Operadores aritméticos

+ soma

- subtração

* multiplicação

/ divisão

// divisão inteira

% resto da divisão

** potência

```
a = 10  
b = 3  
print(a + b)  
print(a / b)  
print(a % b)  
print(a ** b)
```

Operadores relacionais

Comparam valores e produzem True ou False.

`==` igual

`!=` diferente

`>` maior | `<` menor

`>=` maior ou igual | `<=` menor ou igual

```
idade = 18  
print(idade >= 18)  
print(idade == 18)  
print(idade != 20)
```

Operadores lógicos

and → todas as condições precisam ser verdadeiras.

or → pelo menos uma condição precisa ser verdadeira.

not → inverte o resultado lógico.

Eles serão fundamentais nas estruturas condicionais.

```
idade = 20
possui_curso = True

print(idade >= 18 and
      possui_curso)
```

Expressões: pense antes de codificar

Uma expressão combina valores, variáveis e operadores.

Exemplo: calcular média = $(\text{nota1} + \text{nota2}) / 2$.

O programa deve representar corretamente a regra do problema.

Antes de programar, pergunte: quais são as entradas? qual é o processamento? qual é a saída?

```
nota1 = 8
nota2 = 6
media = (nota1 + nota2) / 2
print(media)
```

Mini-desafio — Calculadora

Em duplas, criar um programa que receba dois números.

Exibir: soma, subtração, multiplicação e divisão.

Exibir também o resto da divisão e a potência.

Organizar o código com nomes claros e mensagens para o usuário.

```
a = float(input("Número 1: "))  
b = float(input("Número 2: "))  
  
    print("Soma:", a + b)  
    print("Produto:", a * b)
```

Desafio contextualizado — Suporte de TI

Um técnico precisa calcular o custo de uma manutenção.

Entrada: valor da peça e valor da mão de obra.

Processamento: custo total e valor com 10% de desconto.

Saída: mostrar os valores formatados.

Próxima etapa

Estruturas condicionais: if, elif e else.

Tomada de decisão em programas.

Validação de dados.

Problemas práticos com regras de negócio.

Depois: repetição, funções, estruturas de dados e POO.



A ideia é evoluir de pequenos programas para aplicações de pequeno porte.